

EE267 Spring 2025 Term Project: Swimmer Segmentation and Pose Estimation

1st Rick Vu Truong
M.S. in Artificial Intelligence
San Jose State University
rick.truong@sjsu.edu

I. INTRODUCTION

A. Motivation

Artificial Intelligence (AI) has transformed computer vision, enabling machines to interpret visual data much like humans. Two critical techniques in this field are image segmentation and human pose estimation. Image segmentation isolates objects from their backgrounds, essential for tasks like sports analysis and medical imaging. Human pose estimation, on the other hand, detects body joints and movement, providing valuable insights for elds such as biomechanics and augmented reality.

B. Project Goals

The main objectives we want to accomplish in our project are to:

- 1) *Preprocessing Pipeline*: Create a video processing pipeline to process and transform images frame-by-frame
- 2) *Image Segmentation*: Segment the swimmer from the background using Meta's Segment Anything Model (SAM)
- 3) *Keypoint Generation*: Detect and label the swimmer's joints using a human pose estimation model of your choice (either YOLO or Meta Sapiens)
- 4) *Pose Estimation*: Classify the pose of the swimmer

C. Contributions

I've made various contributions each of which have taken the project one step closer to accomplishing each of the aforementioned objectives.

Firstly, I was fully in charge of project management. I was in charge of establishing the objectives and direction of the project, conducting research to find the necessary tools to implement our systems, setting deadlines, and task delegation.

Thereafter began the implementation phase of my project. I created the video processing pipeline which processed and transformed each swim video frame-by-frame for further image segmentation, pose estimation, and stroke classification. Subsequently, I used Meta SAM's model to perform image segmentation on each frame containing the swimmer in the swimming pool. Afterwards, I used Ultralytics's YOLOv11 Pose Estimation model to find the keypoints of the swimmer from segmented images. With a csv file storing keypoint data of the processed images, I trained multiple classifiers for swim pose estimation.

```
# Advanced preprocessing pipeline for each frame
advanced_frame_transforms = T.Compose([
    T.Resize(389, 389),
    T.CenterCrop(389),

    # Bilateral (edge preserving) smoothing
    T.Lambda(lambda x: K.filters.bilateral_blur(x.unsqueeze(0), kernel_size=3, sigma_color=0.1, sigma_space=(1.0, 1.0)).squeeze(0)),

    # Sobel edge detection
    T.Lambda(lambda x: K.filters.spatial_gradient(x.unsqueeze(0), mode='sobel', order=1).squeeze(0)),

    # Adjust contrast
    T.Lambda(lambda x: K.enhance.adjust_contrast(x.unsqueeze(0), 1.5).squeeze(0)),

    # Image sharpening
    T.Lambda(lambda x: K.enhance.sharpen(x.unsqueeze(0), 2.0).squeeze(0)),

    # Remove blue from image
    T.Lambda(lambda x: remove_blue_from_image(x.unsqueeze(0)).squeeze(0)),

    T.ToType(torch.FloatTensor, scale=True),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]),
])
```

Fig. 1. Image Transformation Pipeline

II. DESIGN

This project's swim analysis system consist of four components: a video preprocessing pipeline, image segmentation component, keypoint generation feature, and pose estimator. I detail the design and architecture of each below.

A. Video Preprocessing Pipeline

The Video Preprocessing Pipeline is a preprocessing pipeline which processes and transforms swimming images frame-by-frame.

The Video Preprocessing Pipeline starts with taking in a file path to a video file. The Video Preprocessing Pipeline opens the video using the OpenCV library, and it is here frame-by-frame processing takes place. For each frame, containing a swimmer and the swimming pool, the Video Preprocessing Pipeline applies a series of image transforms using the torchvision transforms library.

The following depicts the image transforms applies to each frame:

B. Image Segmentation

After applying each of the above transforms to a frame comes Image Segmentation. I used Meta's Segment Anything Model (SAM) to segment the swimmer from the rest of the background. I used the ViT-H SAM model in particular. After downloading Meta's ViT-H SAM model and passing the transformed frames through the model, the swimmer was able to be segmented from the rest of the background in the majority of transformed frames.

C. Keypoint Generation

After segmenting the swimmers from the rest of the background in each frame comes keypoint analysis & generation. In order to perform keypoint analysis and generation, I used Ultralytics's YOLOv11 Pose Estimation model. Upon detection of a human figure in an image, Ultralytics's YOLOv11 Pose Estimation model generates a bounding box capturing the body and 17 x, y coordinates representing the keypoints for a human body:

- 1) Nose
- 2) Left Eye
- 3) Right Eye
- 4) Left Ear
- 5) Right Ear
- 6) Left Shoulder
- 7) Right Shoulder
- 8) Left Elbow
- 9) Right Elbow
- 10) Left Wrist
- 11) Right Wrist
- 12) Left Hip
- 13) Right Hip
- 14) Left Knee
- 15) Right Knee
- 16) Left Ankle
- 17) Right Ankle

From the previous image transformation and segmentation, Ultralytics's YOLOv11 Pose Estimation model readily extracts 17 keypoint coordinates for the swimmer of interest for each frame for which we save into a CSV file for the final stage of the pipeline: Pose Estimation.

D. Pose Estimation

Through several videos that contain multiple swimmers performing the various strokes (back, breast, butterfly, freestyle) that I fed through our transformation, segmentation and keypoint generation pipeline, I was able to accumulate enough keypoint data, stored in a CSV file, to train multiple classifiers for the estimation of swim posture. I trained the following types of model for swim pose estimation:

- 1) Random Forest Classifier
- 2) Support Vector Machine
- 3) Multi-Layer Perceptron
- 4) Convolutional Neural Network
- 5) AlexNet
- 6) ResNet-50

The classification performance and results are detailed in the next section.

III. RESULTS

To estimate the swim pose of the swimmer from the video data, I trained various Random Forest Classifiers, Support Vector Machines, Multi-Layer Perceptrons, Convolutional Neural Networks, pretrained AlexNet model, pretrained ResNet-50 model, and a transformer model. I have the classification

metrics of each architecture below, and a table containing a summary of the top test accuracies of amongst each architecture.

A. Random Forest

I trained 4 Random Forest Classifiers from the keypoint data and recorded the classification performance on the test set below:

- 1) N estimators: 50, minimum sample split: 2, minimum leaf sample size: 1
- 2) N estimators: 100, minimum sample split: 2, minimum leaf sample size: 1
- 3) N estimators: 150, minimum sample split: 2, minimum leaf sample size: 1
- 4) N estimators: 200, minimum sample split: 2, minimum leaf sample size: 1

N_estimators	min sample split	min leaf sample size	Test Acc.
50	2	1	84.97%
100	2	1	86.53%
150	2	1	85.49%
200	2	1	86.01%

TABLE I
TEST ACCURACIES OF RANDOM FOREST CLASSIFIERS WITH DIFFERENT NUMBERS OF ESTIMATORS

B. Support Vector Machine

I trained 4 Support Vector Machines from the keypoint data and recorded the classification performance on the test set below:

- 1) Kernel: radial basis function (rbf), C (Regularization parameter): 0.5
- 2) Kernel: radial basis function (rbf), C (Regularization parameter): 1.0
- 3) Kernel: radial basis function (rbf), C (Regularization parameter): 2.0
- 4) Kernel: polynomial, degree: 3, C (Regularization parameter): 1.0

Kernel	Regularization Parameter	Degree	Test Acc.
rbf	0.5		59.33%
rbf	1.0		64.25%
rbf	2.0		67.36%
poly	1.0	3	59.33%

TABLE II
TEST ACCURACIES OF SUPPORT VECTOR MACHINES WITH DIFFERENT KERNELS & REGULARIZATION

C. Multi-Layer Perceptron

I trained 3 Multi-Layer Perceptron (MLP) architectures from the keypoint data:

- 1) MLP-1: 3 Linear, Batch Normalization, Dropout layers + 1 Linear layer
- 2) MLP-2: 5 Linear, Batch Normalization, Dropout layers + 1 Linear layer

```

Model Architecture:
=====
Input shape: (1, 17, 2)

Layer Structure:
-----
Total trainable parameters: 15,396
-----

flatten: Flatten(start_dim=1, end_dim=-1)

mlp:
0: Linear(in_features=34, out_features=128, bias=True)
1: ReLU()
2: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
3: Dropout(p=0.3, inplace=False)
4: Linear(in_features=128, out_features=64, bias=True)
5: ReLU()
6: BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
7: Dropout(p=0.3, inplace=False)
8: Linear(in_features=64, out_features=32, bias=True)
9: ReLU()
10: BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
11: Dropout(p=0.3, inplace=False)
12: Linear(in_features=32, out_features=4, bias=True)
=====

```

Fig. 2. Detailed MLP-1 Architecture

```

Epoch [500/500], Loss: 0.6842

Test Accuracy: 0.7280

Classification Report:

```

	precision	recall	f1-score	support
back	0.77	0.72	0.75	134
breast	0.66	0.76	0.70	128
fly	0.76	0.82	0.79	67
freestyle	0.80	0.56	0.66	57
accuracy			0.73	386
macro avg	0.75	0.72	0.73	386
weighted avg	0.74	0.73	0.73	386

Fig. 3. MLP-1 Classification Results

3) MLP-3: 8 Linear, Batch Normalization, Dropout layers + 1 Linear layer

Each of these networks were trained on 500 epochs. Their architectures and classification performance on the test set is recorded below.

Architecture	Test Accuracy
3 Linear, Batch Normalization, Dropout layers + 1 Linear layer	68.42%
5 Linear, Batch Normalization, Dropout layers + 1 Linear layer	63.51%
8 Linear, Batch Normalization, Dropout layers + 1 Linear layer	60.17%

TABLE III

TEST ACCURACIES OF MLP ARCHITECTURES WITH VARYING NUMBERS OF LINEAR LAYERS

D. Convolutional Neural Network

Although keypoint data is not image data—where Convolutional Neural Networks (CNNs) are traditionally effective at capturing both high-level and fine-grained features—the (x, y)

```

Model Architecture:
=====
Input shape: (1, 17, 2)

Layer Structure:
-----
Total trainable parameters: 82,084
-----

flatten: Flatten(start_dim=1, end_dim=-1)

mlp:
0: Linear(in_features=34, out_features=128, bias=True)
1: ReLU()
2: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
3: Dropout(p=0.3, inplace=False)
4: Linear(in_features=128, out_features=256, bias=True)
5: ReLU()
6: BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
7: Dropout(p=0.3, inplace=False)
8: Linear(in_features=256, out_features=128, bias=True)
9: ReLU()
10: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
11: Dropout(p=0.3, inplace=False)
12: Linear(in_features=128, out_features=64, bias=True)
13: ReLU()
14: BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
15: Dropout(p=0.3, inplace=False)
16: Linear(in_features=64, out_features=32, bias=True)
17: ReLU()
18: BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
19: Dropout(p=0.3, inplace=False)
20: Linear(in_features=32, out_features=4, bias=True)
=====

```

Fig. 4. Detailed MLP-2 Architecture

```

Epoch [500/500], Loss: 0.6351

Test Accuracy: 0.7461

Classification Report:

```

	precision	recall	f1-score	support
back	0.74	0.77	0.75	134
breast	0.69	0.77	0.73	128
fly	0.81	0.78	0.79	67
freestyle	0.87	0.60	0.71	57
accuracy			0.75	386
macro avg	0.78	0.73	0.75	386
weighted avg	0.75	0.75	0.75	386

Fig. 5. MLP-2 Classification Results

coordinate structure forms a grid-like pattern that CNNs are still well-suited to process.

I trained 3 Convolutional Neural Network (CNN) architectures from the keypoint data:

- 1) CNN-1: 3 1D Convolution, ReLU, 1D Batch Normalization layers + 1 Linear, ReLU, Dropout layer + 1 Linear layer
- 2) CNN-2: 5 1D Convolution, ReLU, 1D Batch Normalization layers + 3 Linear, ReLU, Dropout layer + 1 Linear layer
- 3) CNN-3: 7 1D Convolution, ReLU, 1D Batch Normalization layers + 3 Linear, ReLU, Dropout layer + 1 Linear layer

Each of these networks were trained on 500 epochs. Their architectures and classification performance on the test set is recorded below.

```

Model Architecture:
=====
Input shape: (1, 17, 2)

Layer Structure:
-----
Total trainable parameters: 280,996
-----

flatten: Flatten(start_dim=1, end_dim=-1)

mlp:
 0: Linear(in_features=34, out_features=128, bias=True)
 1: ReLU()
 2: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 3: Dropout(p=0.3, inplace=False)
 4: Linear(in_features=128, out_features=256, bias=True)
 5: ReLU()
 6: BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 7: Dropout(p=0.3, inplace=False)
 8: Linear(in_features=256, out_features=256, bias=True)
 9: ReLU()
10: BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
11: Dropout(p=0.3, inplace=False)
12: Linear(in_features=256, out_features=256, bias=True)
13: ReLU()
14: BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
15: Dropout(p=0.3, inplace=False)
16: Linear(in_features=256, out_features=256, bias=True)
17: ReLU()
18: BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
19: Dropout(p=0.3, inplace=False)
20: Linear(in_features=256, out_features=128, bias=True)
21: ReLU()
22: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
23: Dropout(p=0.3, inplace=False)
24: Linear(in_features=128, out_features=64, bias=True)
25: ReLU()
26: BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
27: Dropout(p=0.3, inplace=False)
28: Linear(in_features=64, out_features=32, bias=True)
29: ReLU()
30: BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
31: Dropout(p=0.3, inplace=False)
32: Linear(in_features=32, out_features=4, bias=True)
=====

```

Fig. 6. Detailed MLP-3 Architecture

```

Epoch [500/500], Loss: 0.6017

Test Accuracy: 0.7332

Classification Report:
      precision    recall  f1-score   support

   back         0.77       0.77       0.77        134
  breast         0.65       0.81       0.72        128
    fly         0.86       0.73       0.79         67
 freestyle         0.73       0.47       0.57         57

 accuracy                   0.73        386
 macro avg         0.75       0.70       0.72        386
 weighted avg         0.74       0.73       0.73        386

```

Fig. 7. MLP-3 Classification Results

Architecture	Test Accuracy
3 1D Convolution, ReLU, 1D Batch Normalization layers + 1 Linear, ReLU, Dropout layer + 1 Linear layer	79.79%
5 1D Convolution, ReLU, 1D Batch Normalization layers + 3 Linear, ReLU, Dropout layer + 1 Linear layer	77.98%
7 1D Convolution, ReLU, 1D Batch Normalization layers + 3 Linear, ReLU, Dropout layer + 1 Linear layer	77.20%

TABLE IV

TEST ACCURACIES OF CNN ARCHITECTURES WITH VARYING NUMBERS OF CONVOLUTIONAL LAYERS

```

Model Architecture:
=====
Input shape: (1, 17, 2)

Layer Structure:
-----
Total trainable parameters: 40,100
-----

conv1:
 0: Conv1d(2, 32, kernel_size=(3,), stride=(1,), padding=(1,))
 1: ReLU()
 2: BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv2:
 0: Conv1d(32, 64, kernel_size=(3,), stride=(1,), padding=(1,))
 1: ReLU()
 2: BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv3:
 0: Conv1d(64, 128, kernel_size=(3,), stride=(1,), padding=(1,))
 1: ReLU()
 2: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

global_pool: AdaptiveAvgPool1d(output_size=1)

fc:
 0: Linear(in_features=128, out_features=64, bias=True)
 1: ReLU()
 2: Dropout(p=0.5, inplace=False)
 3: Linear(in_features=64, out_features=4, bias=True)
=====

```

Fig. 8. Detailed CNN-1 Architecture

E. AlexNet

Additionally, I trained a pretrained AlexNet model on the keypoint data. The classification performance on the test set is recorded below:

```

Epoch [500/500], Loss: 0.3816

Test Accuracy: 0.7979

Classification Report:
      precision    recall  f1-score   support

   back         0.84       0.81       0.82        134
  breast         0.73       0.88       0.80        128
    fly         0.85       0.78       0.81         67
 freestyle         0.82       0.63       0.71         57

 accuracy                   0.80        386
 macro avg         0.81       0.77       0.79        386
 weighted avg         0.80       0.80       0.80        386

```

Fig. 9. CNN-1 Classification Results

```

Model Architecture:
=====
Input shape: (1, 17, 2)

Layer Structure:
-----
Total trainable parameters: 698,148
-----

conv1:
  0: Conv1d(2, 32, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv2:
  0: Conv1d(32, 64, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv3:
  0: Conv1d(64, 128, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv4:
  0: Conv1d(128, 256, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv5:
  0: Conv1d(256, 512, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

global_pool: AdaptiveAvgPool1d(output_size=1)

fc:
  0: Linear(in_features=512, out_features=256, bias=True)
  1: ReLU()
  2: Dropout(p=0.5, inplace=False)
  3: Linear(in_features=256, out_features=128, bias=True)
  4: ReLU()
  5: Dropout(p=0.5, inplace=False)
  6: Linear(in_features=128, out_features=64, bias=True)
  7: ReLU()
  8: Dropout(p=0.5, inplace=False)
  9: Linear(in_features=64, out_features=4, bias=True)
=====

```

Fig. 10. Detailed CNN-2 Architecture

Test Accuracy: 0.7798

Classification Report:				
	precision	recall	f1-score	support
back	0.82	0.76	0.79	134
breast	0.70	0.85	0.77	128
fly	0.87	0.82	0.85	67
freestyle	0.81	0.61	0.70	57
accuracy			0.78	386
macro avg	0.80	0.76	0.78	386
weighted avg	0.79	0.78	0.78	386

Fig. 11. CNN-2 Classification Results

F. ResNet-50

I trained a pretrained ResNet-50 model on the keypoint data. The classification performance on the test set is recorded below:

IV. ANALYSIS & CONCLUSION

While further testing could have been carried out in image transformation, alternative image segmentation models could have been evaluated, various keypoint generation models could have been employed, different hyperparameters could have

```

Model Architecture:
=====
Input shape: (1, 17, 2)

Layer Structure:
-----
Total trainable parameters: 698,148
-----

conv1:
  0: Conv1d(2, 32, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv2:
  0: Conv1d(32, 64, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv3:
  0: Conv1d(64, 128, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv4:
  0: Conv1d(128, 256, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

conv5:
  0: Conv1d(256, 512, kernel_size=(3,), stride=(1,), padding=(1,))
  1: ReLU()
  2: BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

global_pool: AdaptiveAvgPool1d(output_size=1)

fc:
  0: Linear(in_features=512, out_features=256, bias=True)
  1: ReLU()
  2: Dropout(p=0.5, inplace=False)
  3: Linear(in_features=256, out_features=128, bias=True)
  4: ReLU()
  5: Dropout(p=0.5, inplace=False)
  6: Linear(in_features=128, out_features=64, bias=True)
  7: ReLU()
  8: Dropout(p=0.5, inplace=False)
  9: Linear(in_features=64, out_features=4, bias=True)
=====

```

Fig. 12. Detailed CNN-3 Architecture

Epoch [500/500], Loss: 0.2612

Test Accuracy: 0.7720

Classification Report:				
	precision	recall	f1-score	support
back	0.75	0.78	0.76	134
breast	0.73	0.83	0.77	128
fly	0.94	0.75	0.83	67
freestyle	0.78	0.67	0.72	57
accuracy			0.77	386
macro avg	0.80	0.75	0.77	386
weighted avg	0.78	0.77	0.77	386

Fig. 13. CNN-3 Classification Results

Architecture	Test Accuracy
AlexNet	78.50%

TABLE V
TEST ACCURACY OF ALEXNET

Architecture	Test Accuracy
ResNet-50	78.50%

TABLE VI
TEST ACCURACY OF RESNET-50

```
Epoch [500/500], Loss: 0.1964

Test Accuracy: 0.7850

Classification Report:

```

	precision	recall	f1-score	support
back	0.80	0.80	0.80	134
breast	0.78	0.78	0.78	128
fly	0.83	0.79	0.81	67
freestyle	0.73	0.75	0.74	57
accuracy			0.78	386
macro avg	0.78	0.78	0.78	386
weighted avg	0.79	0.78	0.79	386

Fig. 14. AlexNet Classification Results

been explored, and diverse ML models/architectures could have been investigated, the main insights from my experiments were as follows:

- 1) Traditional Random Forest Classifier performed best for swim pose estimation
- 2) Despite keypoint data not being traditional image data that CNNs excel in processing, pretrained AlexNet, ResNet, and various CNNs outperformed MLPs in processing this data, achieving superior results in swim pose estimation.

A traditional Random Forest Classifier outperforming the various convolutional neural network architectures for swim pose estimation does at first seem perplexing. However, through a closer examination of the keypoint data each model was trained on, one can see that the keypoint data is sparse and has a large number of keypoints which were not detected

```
Epoch [500/500], Loss: 0.2250

Test Accuracy: 0.7694

Classification Report:

```

	precision	recall	f1-score	support
back	0.76	0.80	0.78	134
breast	0.75	0.80	0.77	128
fly	0.79	0.78	0.78	67
freestyle	0.82	0.63	0.71	57
accuracy			0.77	386
macro avg	0.78	0.75	0.76	386
weighted avg	0.77	0.77	0.77	386

Fig. 15. ResNet-50 Classification Results

Model Class	Top Test Accuracy
Random Forest	86.53%
Support Vector Machine	67.36%
MLP	68.42%
CNN	79.79%
AlexNet	78.50%
ResNet-50	76.94%

TABLE VII
TEST ACCURACIES OF MOST PERFORMANT MODEL FOR EACH MODEL CLASS

(x,y coordinates default to 0,0). The Random Forest Classifier could have handled this sparse data better than the CNNs did which are more effective at handling rich image data.

The second insight where CNNs outperformed MLPs in processing keypoint data for further swim pose estimation is also interesting, but could be due to a similar phenomena which arises from sparse data. When processing sparse data, MLP performance may suffer as parameters and weights through the network could be updated towards 0. However, through convolution, the sparseness of this data may be 'averaged' out, especially with larger kernel windows.

In conclusion, from video data of multiple swimmers performing various strokes, I was able to train various classifiers predicting the stroke of the swimmer with up to 86% accuracy. For future work, I would love to experiment with how transformer models would perform on this task. I was unfortunately unable to due to resource and time constraints.

REFERENCES

- [1] "Transforms v2: End-to-end object detection/segmentation example — Torchvision main documentation," Pytorch.org, 2024. https://docs.pytorch.org/vision/master/auto_examples/transforms/plot_transforms_e2e.html (accessed May 09, 2025).
- [2] M. Patel, "The Complete Guide to Image Preprocessing Techniques in Python," Medium, Oct. 23, 2023. <https://medium.com/@maahip1304/the-complete-guide-to-image-preprocessing-techniques-in-python-dca30804550c>
- [3] "Segment Anything," GitHub, Apr. 17, 2023. <https://github.com/facebookresearch/segment-anything>
- [4] Ultralytics, "Pose," docs.ultralytics.com. <https://docs.ultralytics.com/tasks/pose/>